CODEALPHA INTERNSHIP

TASK 4: MINI PROJECT PROPOSAL

IoT in Robotics and Automation

Design of an IoT-Enabled Smart Monitoring and Control System

Submitted by	Rafiqa Begum
Domain	Robotics & Automation
Project type	Mini Project Proposal
Organization	CodeAlpha

Project focus: Connecting sensors, a microcontroller, robotic/automation equipment, and a cloud dashboard so that operating conditions can be monitored and selected actions can be controlled remotely.

Submitted in partial fulfilment of the CodeAlpha Robotics & Automation internship task requirements.

ABSTRACT

The Internet of Things (IoT) connects physical devices to networks so that data can be collected, shared, visualized, and used to support decisions. In robotics and automation, IoT can connect sensors, controllers, actuators, machines, and cloud platforms. This proposal presents an IoT-enabled smart monitoring and control system suitable for a small robotic or automated workstation. A microcontroller such as an ESP32 reads sensor values, communicates through Wi-Fi, and sends measurements to a cloud dashboard. An operator can view trends and receive alerts when a measured value crosses a defined limit. Where appropriate and safely designed, the operator can also issue a limited command to an actuator through a protected control interface. The proposed system can support condition monitoring, simple pick-and-place work cells, smart conveyors, laboratory robots, and educational prototypes. The proposal describes the objectives, architecture, components, workflow, implementation plan, safety considerations, benefits, limitations, and possible future enhancements.

1. INTRODUCTION

Robotics and automation are used to perform repetitive, precise, or hazardous tasks in manufacturing, healthcare, agriculture, logistics, and homes. Traditional automation often relies on local controllers and displays. Although these systems can operate effectively, information may be difficult to access when a supervisor is away from the machine or when several machines must be monitored together.

IoT adds network connectivity to sensors and controllers. Machine data can be sent to a dashboard, reviewed over time, and used to identify unusual operating conditions. For example, a temperature sensor can monitor a motor enclosure, a proximity sensor can count objects on a conveyor, and a current sensor can help indicate whether a motor is drawing an unexpected amount of power. The controller remains responsible for immediate, local control; the cloud is used mainly for visualization, records, and higher-level monitoring.

This project proposes a small, modular system that demonstrates the relationship between sensing, embedded control, communication, and automation. It is a design proposal and can be implemented as a prototype using low-voltage educational hardware.

2. PROBLEM STATEMENT AND OBJECTIVES

Problem statement

In a basic robotic or automated setup, an operator may have to stand beside the equipment to observe sensor values and operating status. There may be no convenient

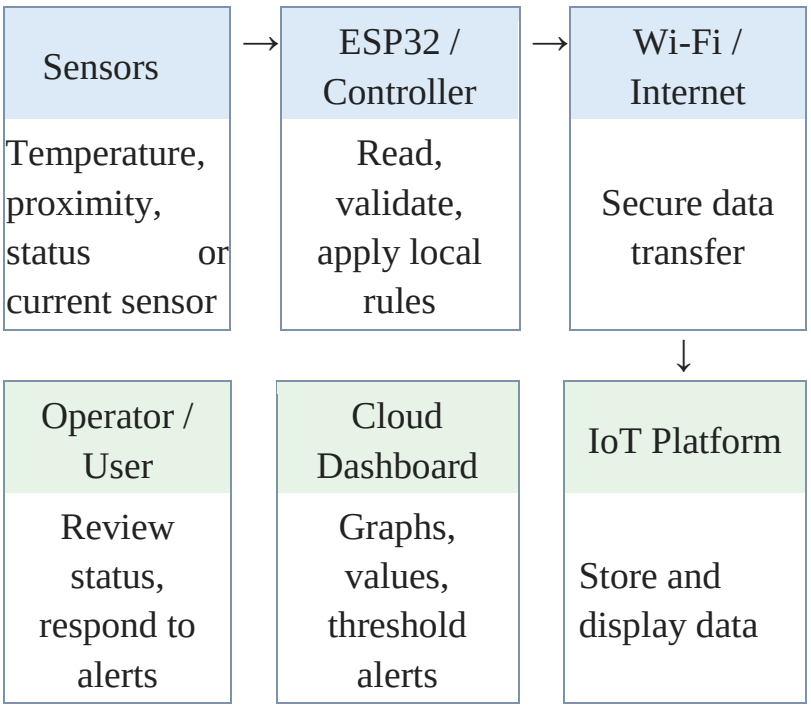
way to review past measurements or receive a notification when a parameter becomes abnormal. A low-cost IoT monitoring layer can make this information easier to access without replacing the machine's local safety and control systems.

Objectives

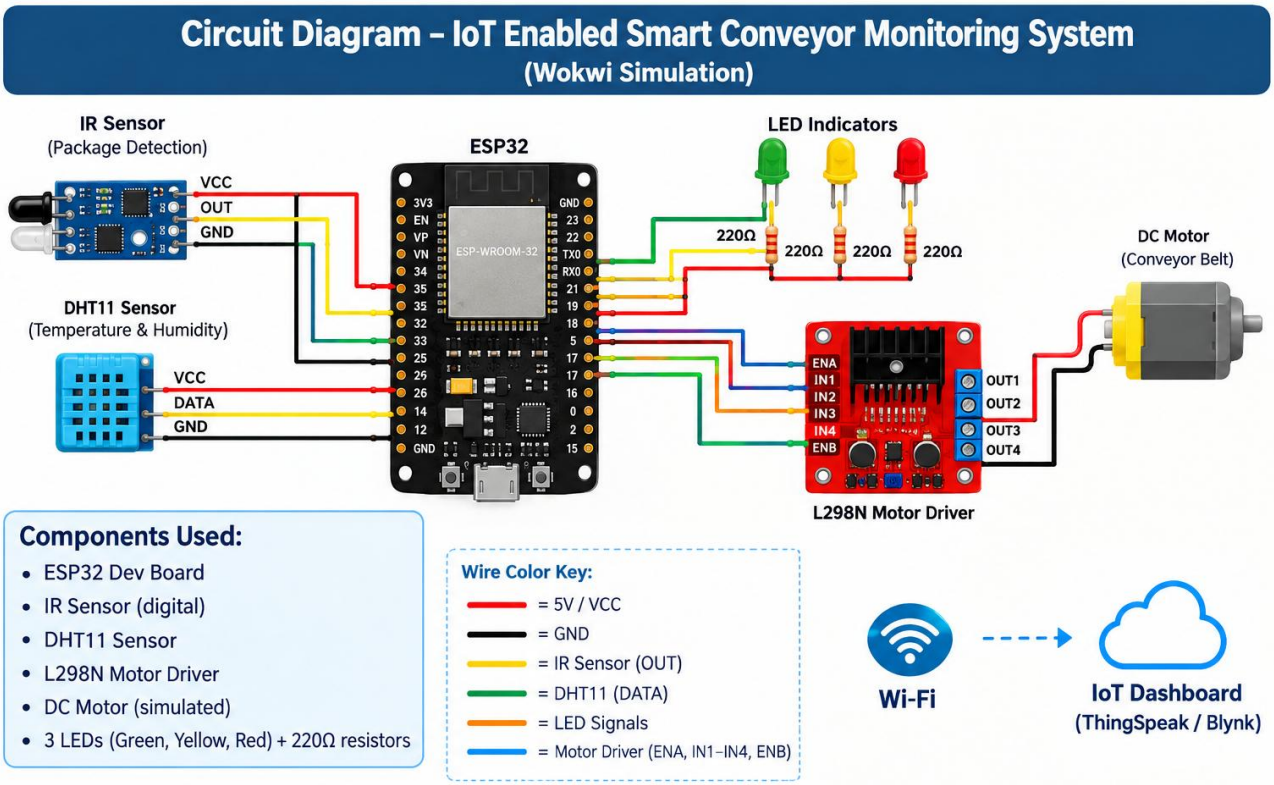
- Design an architecture that connects sensors and a controller to an IoT dashboard.
- Monitor selected parameters such as temperature, humidity, object count, or motor status.
- Display current values and historical trends through a cloud dashboard.
- Define threshold-based alerts for selected abnormal readings.
- Demonstrate how IoT data can support maintenance, productivity, and remote supervision.
- Identify security and safety measures for any connected control function.

3. PROPOSED SYSTEM AND BLOCK DIAGRAM

The system has five main layers: sensing, local processing, communication, cloud visualization, and operator interaction. A sensor provides a signal to the microcontroller. The controller checks the value, applies simple rules, and transmits the data over Wi-Fi. A cloud service stores or displays the measurements. The operator views the dashboard and responds to alerts. If a control feature is added, it should be restricted to safe, low-voltage actions and should not bypass emergency stops or machine interlocks.



SYSTEM BLOCK DIAGRAM



Example operating sequence

- A sensor measures a physical condition or detects an object.
- The ESP32 reads the sensor and checks whether the measurement is valid.
- The controller applies a local threshold rule and updates the local status indicator if included.
- The reading and timestamp are sent to an IoT service through Wi-Fi.
- The dashboard plots the data and highlights values outside the configured range.
- The operator reviews the alert and follows the approved operating procedure.

4. HARDWARE AND SOFTWARE REQUIREMENTS

Item	Purpose
ESP32 development board	Reads sensors and provides Wi-Fi connectivity.
DHT11/DHT22 sensor (optional)	Measures ambient temperature and humidity.
IR/proximity sensor	Detects objects or counts items on a small conveyor.
Low-voltage LED/buzzer	Provides a local status or alert indication.
Motor/servo (optional)	Demonstrates a small low-voltage actuator; use a suitable driver.
Breadboard, jumper wires, USB supp	lyPrototype wiring and safe power for the controller.
Computer and Wi-Fi access	Programming, configuration, and network access.
Arduino IDE or compatible tool	Develops and uploads the controller program.
ThingSpeak or similar IoT platform	Displays measurements and trends; service limits may apply.

5. METHODOLOGY AND WORKING PRINCIPLE

Step 1 — Define the use case

Choose a small automation scenario, such as monitoring a motor enclosure or counting packages on a conveyor. Decide which values are important and set reasonable demonstration thresholds. Thresholds should be chosen for the actual sensor and equipment rather than copied without checking.

Step 2 — Connect the sensors

Connect the selected sensor to the controller according to its datasheet. Confirm voltage compatibility and pin assignments before powering the circuit. For motors or other

loads, use a properly rated driver and a separate supply when required; do not power a motor directly from a microcontroller GPIO pin.

Step 3 — Program local processing

Write a controller program to initialize the sensor, read data at a suitable interval, reject clearly invalid readings, and determine whether a threshold has been crossed. Keep essential stop conditions and safety interlocks local so they continue to function even if Wi-Fi or the cloud service becomes unavailable.

Step 4 — Configure the IoT dashboard

Create a channel or data source on an IoT platform, define fields for the selected measurements, and configure the controller with the platform's connection details. Send readings with timestamps where supported. Create a graph for each field and an alert rule for selected threshold violations.

Step 5 — Test the system

Compare sensor readings with a reasonable reference, check that the dashboard updates, and confirm that alert rules work as expected. Test what happens when Wi-Fi is disconnected. The controller should continue its safe local behavior, buffer or discard data according to the design, and reconnect without causing unexpected actuator movement.

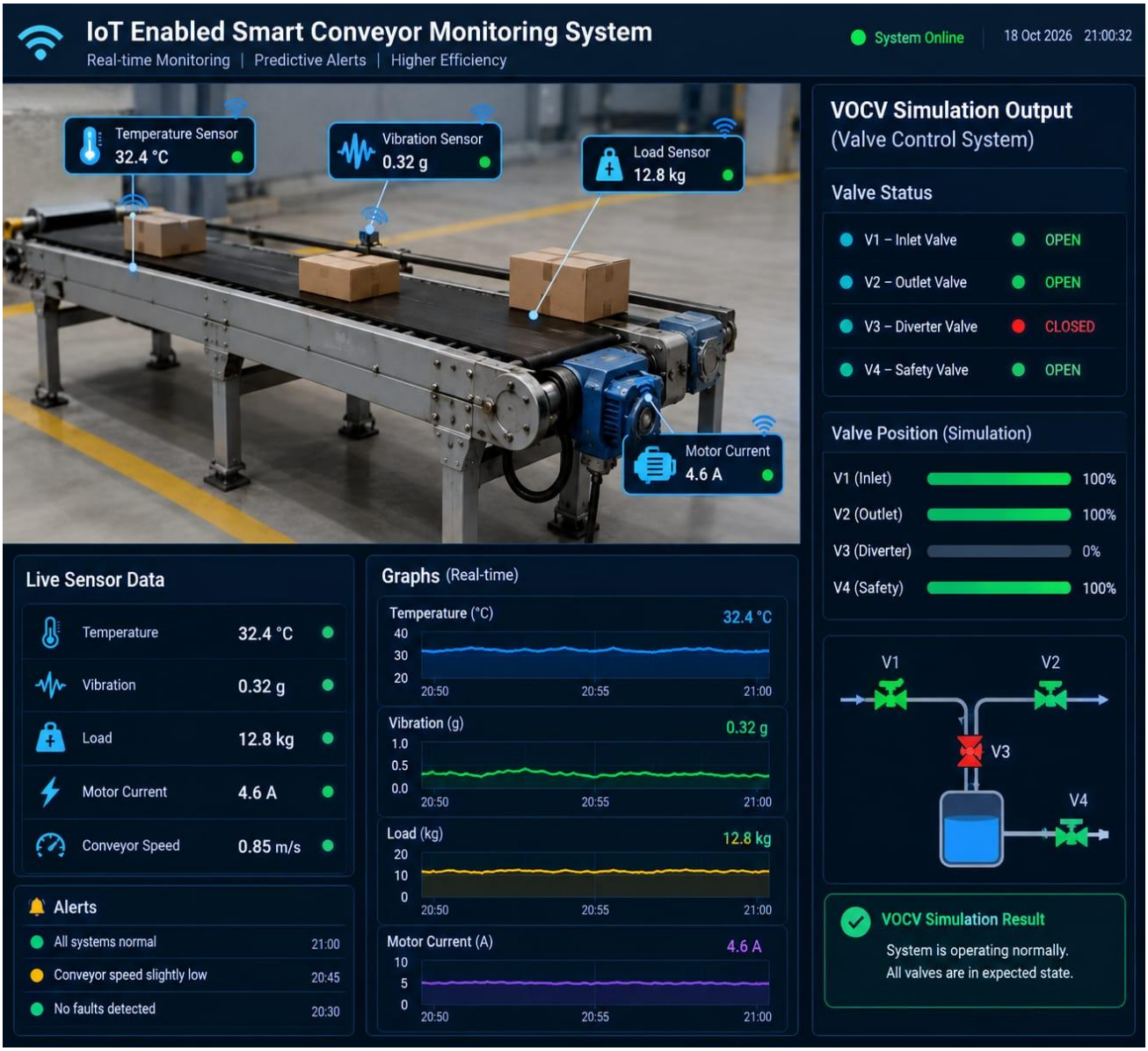
6. EXPECTED OUTPUTS AND EVALUATION

Expected outputs

- A working or simulated sensor-to-controller connection.
- A cloud dashboard showing live or periodically updated measurements.
- Graphs that help compare values over time.
- A threshold alert or local indicator for an out-of-range condition.
- A short test record containing normal readings, test readings, and observed dashboard behavior.

Evaluation criteria

The prototype can be evaluated by checking sensor reliability, update consistency, alert behavior, ease of use, and safe response to network failure. Record actual results during implementation; do not report assumed values as measured experimental results.



7. APPLICATIONS

- **Manufacturing:** monitoring machine temperature, operating status, and production counts.
- **Conveyor systems:** detecting packages and recording throughput.
- **Educational robotics:** viewing sensor data while learning embedded systems and networking.
- **Smart agriculture:** connecting environmental sensors to automated irrigation equipment.
- **Healthcare support equipment:** monitoring non-critical operating parameters under appropriate safety and privacy controls.

8. ADVANTAGES, LIMITATIONS, AND SAFETY

Advantages

- Remote visibility of operating data and trends.
- Low-cost prototyping with widely available microcontrollers and sensors.
- Easier record keeping and faster awareness of selected abnormal conditions.
- A modular design that can be expanded with additional sensors.

Limitations

- The system depends on sensor quality, network availability, and platform service limits.
- Cloud updates may have latency and are not suitable for time-critical control loops.
- Incorrect thresholds or poorly calibrated sensors can generate false alerts.
- A prototype does not replace industrial safety engineering or a certified control system.

Future scope

The design can be expanded with additional sensors, a mobile-friendly dashboard, device-health reporting, and a database for longer-term records. A camera or machine-vision module could be added for object classification, while a local gateway could support multiple devices. Data analysis could help identify patterns, but any predictive model should be trained and validated using representative data. Industrial deployment would require a threat model, access control, secure updates, robust networking, and a formal safety review.

10. CONCLUSION

This proposal demonstrates how IoT can improve visibility and supervision in robotics and automation. By combining sensors, an ESP32-class controller, Wi-Fi, and a cloud dashboard, an operator can review measurements, observe trends, and receive alerts when selected conditions occur. The main design principle is to use the cloud for monitoring and high-level information while retaining immediate control and safety functions locally. The system is modular, affordable for education, and suitable as a starting point for a small prototype. Practical implementation should include careful wiring, testing, secure handling of credentials, and honest documentation of measured results.

REFERENCES

1. Espressif Systems, “ESP32 Series Datasheet.” https://www.espressif.com/en/support/documents/technical-documents
2. MathWorks, “ThingSpeak Documentation.” https://www.mathworks.com/help/thingspeak/
3. Arduino, “Arduino Documentation.” https://docs.arduino.cc/